

BAN404 samlet cheatsheet

Sammenslått fra eksamen-, modellkort- og oversiktlig-versjonene; kontrollert mot Foreleser/

2026-05-18

0. Eksamen og forelesers stil

Hva som går igjen.

- Forklar hva kode gjør, nok linje for linje til at objektiv, dataflyt og modellvalg er tydelig.
- Start ofte med deskriptiv statistikk: riktig tabell/figur for variabeltypene og konkrete funn med tall.
- Bruk treningsdata til modellvalg, kryssvalidering og terskelvalg; bruk testdata til endelig evaluering.
- For rene deskriptive eller forklarende spørsmål kan full data forsvares, men skriv hvorfor.
- Ikke bruk variabler som ikke er kjent når prediksjonen skal gjøres, eller som lekker fasiten.
- Ved klassifikasjon: ikke bare accuracy. Bruk krysstabell og gjerne rad-proporsjoner.
- En enkel modell som svarer presist på oppgaven er bedre enn en komplisert modell uten god evaluering.

Kontrollert mot forelesers filer.

- Foreleser/02/ex38.ipynb: `statsmodels`, `ISLP.models.ModelSpec`, prediksjons- og konfidensintervall.
- Foreleser/03/ex413_a_d.py, Foreleser/04/ex413_e_g.py, Foreleser/05/ex55.py: logistisk regresjon, LDA/QDA/KNN, krysstabeller og terskler.
- Foreleser/07/ex611.py: OLS, RidgeCV, LassoCV, 50/50-split med mask.
- Foreleser/08/ex76*.ipynb: polynom, step functions og validation set.
- Foreleser/09/gam_example.py: GAM-intuisjon og backfitting med KNN-smoother.
- Foreleser/10-11/ex88*.ipynb: trær, pruning, bagging og random forest.
- Foreleser/12/ex97.py: SVM med GridSearchCV.
- Foreleser/13/ames.py: dummies, MAE, ridge/lasso, trær, boosting og MLPRegressor.
- Foreleser/14/ex128_129_starting_code.qmd: PCA og hierarkisk clustering-importer.

Viktig justering. Forelesers `ames.py` bruker `scaler.fit_transform(X_test)` etter å ha standardisert `train`. Metodisk riktig praksis er `scaler.transform(X_test)`: lær middelverdi og standardavvik på `train`, bruk samme transformasjon på `test`.

Fundamentals. Vi observerer ofte $Y = f(X) + e$ og estimerer f fra data.

- Parametrisk modell: antar form, f.eks. lineær modell. Få parametere, lav varians, men kan ha høy bias hvis formen er feil.
- Ikke-parametrisk modell: få formantakelser, f.eks. KNN, trær og splines. Fleksibelt, men kan få høy varians.

- Testfeil kan tenkes som $\text{bias}^2 + \text{variance} + \text{irreducible error}$.
- Mer fleksibilitet gir ofte lavere treningsfeil, men testfeil kan øke på grunn av overfitting.

Imports du ofte trenger.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import statsmodels.api as sm
import statsmodels.formula.api as smf
import sklearn.linear_model as skl
from sklearn.model_selection import train_test_split, GridSearchCV,
    KFold
from sklearn.preprocessing import StandardScaler
from ISLP.models import ModelSpec as MS, summarize
```

1. Data, split og evaluering

Forelesers 50/50-split.

```
np.random.seed(1234)
n = len(df)
ntrain = n // 2
ind = np.random.choice(np.arange(n), size=ntrain, replace=False)
mask = np.ones(n, dtype=bool)
mask[ind] = False

train = df.iloc[ind]
test = df.iloc[mask]
```

Ikke bruk `~ind` når `ind` er heltallsindekser. Bruk boolsk `mask`, `df.drop(ind)` når indeksen passer, eller:

```
test_idx = np.setdiff1d(np.arange(n), ind)
```

Alternativ foreleserstil med `sklearn`.

```
train, test = train_test_split(df, test_size=0.5, random_state=123)
```

Forklaringsvariabler og respons. X er prediktorer/forklaringsvariabler. y er responsen.

```
X = pd.get_dummies(df.drop(columns="y"), drop_first=True)
X = X.astype(float).fillna(0)
y = df["y"]
```

For binær tekstrespons:

```
y = (df["Direction"] == "Up").astype(int)
```

Metrikker.

```
RSS = np.sum((y - pred)**2)
TSS = np.sum((y - y.mean())**2)
MSE = np.mean((y - pred)**2)
RMSE = np.sqrt(MSE)
MAD = np.mean(np.abs(y - pred)) # foreleser bruker ofte MAE/MAD
R2 = 1 - RSS / TSS
accuracy = np.mean(y_true == pred)
```

Krysstabell.

```
cm = pd.crosstab(y_true, pred,
                 rownames=["Actual"], colnames=["Predicted"])
row_props = pd.crosstab(y_true, pred, normalize="index")
```

Hvis rader er faktisk og kolonner predikert:

	Pred 0	Pred 1
Actual 0	TN	FP
Actual 1	FN	TP

Sensitivitet for klasse 1 = $TP / (TP + FN)$. Spesifisitet for klasse 0 = $TN / (TN + FP)$.

2. Deskriptiv statistikk

Velg figur etter variabeltyper.

- Kontinuerlig Y, kontinuerlig X: scatterplot.
- Kontinuerlig Y, kategorisk X: boxplot.
- Kategorisk Y, kontinuerlig X: boxplot av X gruppert etter Y.
- Kategorisk Y, kategorisk X: krystabell, gjerne `normalize="index"`.

```
df.describe(include="all")
df["y"].value_counts(normalize=True)
pd.crosstab(df["cat"], df["y"], normalize="index")
df.boxplot("x", by="y")
plt.scatter(df["x"], df["y"])
pd.plotting.scatter_matrix(df[["x1", "x2", "x3"]])
```

Eksamensformulering. “Jeg bruker treningsdata til deskriptiv analyse fordi senere modeller skal sammenlignes på samme testsett” eller “Jeg bruker full data fordi spørsmålet er deskriptivt/forklarende, ikke prediksjon”.

3. OLS og statsmodels

Modell. $y = \beta_0 + \beta_1 x_1 + \dots + \beta_p x_p + e$. OLS minimerer $\sum_i (y_i - \hat{y}_i)^2$.

Manuell løsning.

```
Xc = np.column_stack([np.ones(len(y)), X])
b = np.linalg.solve(Xc.T @ Xc, Xc.T @ y)
pred = Xc @ b
```

Statsmodels uten formel.

```
X_train_const = sm.add_constant(X_train)
ols = sm.OLS(y_train, X_train_const).fit()
print(ols.summary())
```

```
X_test_const = sm.add_constant(X_test)
pred = ols.predict(X_test_const)
```

ISLP ModelSpec, slik foreleser gjør i Auto-laben.

```
design = MS(["horsepower"]).fit(Auto)
X = design.transform(Auto)
y = Auto["mpg"]
res = sm.OLS(y, X).fit()
summarize(res)
```

```
X_new = design.transform(pd.DataFrame({"horsepower": [98]}))
res.predict(X_new)
res.get_prediction(X_new).conf_int(alpha=0.05) # KI for
↪ E[Y|X]
res.get_prediction(X_new).conf_int(obs=True, alpha=0.05) # PI for ny
↪ observasjon
```

Tolkning.

- Positiv koeffisient: høyere x henger sammen med høyere forventet y, gitt de andre variablene.
- Lav p-verdi: prediktoren er assosiert med Y gitt modellen.
- Konfidensintervall: usikkerhet om forventet respons.
- Prediksjonsintervall: usikkerhet om ny observasjon og er bredere.

4. Ridge og LASSO

Ridge-regresjon. Minimerer $\text{RSS} + \alpha * \sum(\beta_j^2)$. Krymper koeffisienter, setter dem normalt ikke eksakt til null.

LASSO-regresjon. “Least absolute shrinkage and selection operator”. Minimerer $\text{RSS} + \alpha * \sum(\text{abs}(\beta_j))$. Kan sette koeffisienter lik null og fungerer som variabelseleksjon.

Bias-variance. Høy alpha gir enklere modell, høyere bias og lavere varians. Lav alpha gir mer fleksibel modell, lavere bias og høyere varians.

Forelesers ex611.py-mønster.

```
alphas = np.logspace(-4, 4, 200)
```

```
ridge_cv = skl.RidgeCV(alphas=alphas)
ridge_cv.fit(Xtrain, ytrain)
ridgemin = skl.Ridge(alpha=ridge_cv.alpha_)
ridgemin.fit(Xtrain, ytrain)
pred_ridge = ridgemin.predict(Xtest)
```

```
lasso_cv = skl.LassoCV(alphas=alphas)
lasso_cv.fit(Xtrain, ytrain)
lasso = skl.Lasso(alpha=lasso_cv.alpha_)
lasso.fit(Xtrain, ytrain)
pred_lasso = lasso.predict(Xtest)
```

Koeffisientsammenligning.

```
ols_coef = ols.params
ridge_coef = np.concatenate((ridgemin.intercept_, ridgemin.coef_))
lasso_coef = np.concatenate((lasso.intercept_, lasso.coef_))
pd.DataFrame({"OLS": ols_coef, "Ridge": ridge_coef, "Lasso":
↪ lasso_coef})
```

Standardisering når skala betyr noe.

```
scaler = StandardScaler(with_mean=True, with_std=True)
X_train_s = scaler.fit_transform(X_train) # fit kun på train
X_test_s = scaler.transform(X_test)
```

Foreleser standardiserer i `ames.py` for ridge/lasso og neural networks. Metodisk: bruk aldri testdata til å lære scaler.

5. Ridge fra scratch og LOO

Eksamen 2025-funksjonen er Ridge. Den demeaner y og hver kolonne i X, men standardiserer ikke. Med `la=0` blir det OLS på sentrert designmatrise uten separat intercept.

```
def f(b1, X, y, la):
    return np.sum((y - X @ b1)**2) + la * np.sum(b1**2)
```

```
def g(X, y, la):
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    p = Xd.shape[1]
    A = Xd.T @ Xd + la * np.eye(p)
    b1 = np.linalg.solve(A, Xd.T @ yd)
    return b1
```

`f` beregner RSS pluss Ridge-straff. `g` finner koeffisientene som minimerer dette for gitt `la`. Intercept håndteres via `mean(y)` fordi data sentreres.

Prediksjon med samme sentrering.

```
b0 = y.mean()
Xd = X - X.mean(axis=0)
b1 = g(X, y, la=1000)
pred = b0 + Xd @ b1
```

Leave-one-out for optimal la.

```
def loo_ridge(la, X, y):
    n = len(y)
    b0 = y.mean()
    yd = y - b0
    Xd = X - X.mean(axis=0)
    ydpred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        b1 = g(X[idx], y[idx], la)
        ydpred[i] = Xd[i] @ b1
    return np.mean((yd - ydpred)**2)
```

```
lavec = np.linspace(4500, 5000, 10)
testMSE = np.array([loo_ridge(la, X, y) for la in lavec])
best_la = lavec[np.argmin(testMSE)]
```

6. Kryssvalidering og bootstrap

Leave-one-out. Tren modellen n ganger. Hver gang holdes én observasjon utenfor og predikeres.

```
def loo_score(model_func, X, y):
    n = len(y)
    pred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        pred[i] = model_func(X[idx], y[idx])
    return np.mean((y - pred)**2)
```

GridSearchCV.

```
grid = GridSearchCV(model, param_grid, cv=5)
grid.fit(X_train, y_train)
best = grid.best_estimator_
```

Bootstrap. Trekk n observasjoner med tilbakelegging, beregn statistikk, gjenta B ganger.

```
np.random.seed(42)
n = len(y)
B = 10000
vboot = np.zeros(B)

for b in range(B):
    yboot = np.random.choice(y, size=n, replace=True)
    vboot[b] = np.var(yboot, ddof=1)

ci = np.percentile(vboot, [2.5, 97.5])
plt.hist(vboot, bins=40)
plt.axvline(np.var(y, ddof=1), color="black", linestyle="--")
```

Bruk $ddof=1$ for stikkprøvevarians. Persentil-KI: 2.5 og 97.5 prosentil.

7. Logistisk regresjon og terskel

Modell. $P(Y=1|X) = 1/(1+\exp(-(b_0 + bX)))$. Estimeres med maximum likelihood, ikke OLS. Koeffisientene er lineære i log-odds.

Statsmodels GLM.

```
X_train = sm.add_constant(train[["Lag2"]])
y_train = (train["Direction"] == "Up").astype(int)
m = sm.GLM(y_train, X_train, family=sm.families.Binomial()).fit()
```

```
X_test = sm.add_constant(test[["Lag2"]])
prob = m.predict(X_test)
pred = (prob > 0.5).astype(int)
```

Formelvarianten foreleser bruker.

```
df["y"] = (df["Direction"] == "Up").astype(int)
m = smf.logit("y ~ Volume + Lag1 + Lag2 + Lag3 + Lag4 + Lag5",
              data=train).fit()
prob = m.predict(test)
pred = prob > 0.5
```

Terskel ved ubalanserte klasser. Velg terskel på treningsdata, evaluer på testdata.

```
prob_tr = m.predict(train)
for c in [0.01, 0.02, 0.05, 0.10, 0.50]:
    pred_tr = prob_tr > c
    print(c, pd.crosstab(train["y"], pred_tr, normalize="index"))
```

```
prob_te = m.predict(test)
pred_te = prob_te > chosen_cutoff
pd.crosstab(test["y"], pred_te, normalize="index")
```

8. LDA, QDA, Naive Bayes og KNN

LDA/QDA/Naive Bayes.

- LDA antar omtrent normalfordelte prediktorer og lik kovarians i klassene. Stabil ved lite data.
- QDA tillater ulike kovarianser per klasse. Mer fleksibel, trenger mer data.
- Naive Bayes antar prediktorer uavhengige gitt klasse.
- Alle estimerer posterior sannsynlighet $P(Y=k|X)$.

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.naive_bayes import GaussianNB
```

```
lda = LinearDiscriminantAnalysis()
lda.fit(X_train, y_train)
prob = lda.predict_proba(X_test)
pred = np.where(prob[:, 1] > 0.54, "Up", "Down")
```

```
qda = QuadraticDiscriminantAnalysis()
qda.fit(X_train, y_train)
```

```
nb = GaussianNB()
nb.fit(X_train, y_train)
```

Foreleser justerer terskel i Weekly-eksempelet, f.eks. 0.54 for LDA og 0.55 for QDA.

KNN. Regresjon: gjennomsnitt av y for de K nærmeste. Klassifikasjon: andel/majoritet blant naboer. Lav K = fleksibel/høy varians. Høy K = glatt/høy bias. Standardiser ved flere prediktorer.

```
def knn(x0, x, y, K=3):
    d = np.abs(x - x0)
    idx = np.argsort(d)[:K]
    return np.mean(y[idx])
```

Sklearn KNN.

```
from sklearn.neighbors import KNeighborsClassifier, KNeighborsRegressor
```

```
knn_clf = KNeighborsClassifier(n_neighbors=3)
knn_clf.fit(X_train, y_train)
prob = knn_clf.predict_proba(X_test)[:, 1]
pred = prob > 0.5
```

Flere prediktorer.

```
d = np.sqrt(np.sum((X - x0)**2, axis=1))
```

9. Lokal lineær regresjon, eksamen 2024

2024-funksjonen er KNN-lignende, men gjør lokal lineær regresjon på de K nærmeste i stedet for å ta gjennomsnitt.

```
def local_lm(x0, x, y, K=3):
    d = np.abs(x - x0)
    o = np.argsort(d)[:K]
    x1 = x[o]
    y1 = y[o]
    X1 = sm.add_constant(x1)
    reg = sm.OLS(y1, X1).fit()
    return reg.predict([1, x0])[0]
```

LOO for valg av K .

```
def loo_K(K, x, y):
    n = len(x)
    ypred = np.zeros(n)
    for i in range(n):
        idx = np.arange(n) != i
        ypred[i] = local_lm(x[i], x[idx], y[idx], K)
    return np.mean((y - ypred)**2)
```

```
Ks = range(2, 11) # lokal lm trenger minst K=2
mse = np.array([loo_K(K, x, y) for K in Ks])
best_K = list(Ks)[np.argmin(mse)]
```

10. Polynom, step functions og splines

Polynomregresjon. Grad K gir basis $1, x, x^2, \dots, x^K$.

```
def polyreg(k, data):
    return smf.ols(
        f"wage ~ np.vander(age, {k+1}, increasing=True)",
        data=data
    ).fit()
```

```
MSE = np.zeros(20)
for k in range(1, 21):
    reg = polyreg(k, train)
    pred = reg.predict(test)
    MSE[k - 1] = np.mean((test["wage"] - pred)**2)
best_k = np.argmin(MSE) + 1
```

Step function.

```
def stepreg(k, data):
    data = data.copy()
    data["age_cut"] = pd.cut(data["age"], k)
    return smf.ols("wage ~ age_cut", data=data).fit()
```

Kubisk spline fra scratch.

```
def h(x, xi):
    return np.maximum((x - xi)**3, 0)

X_sp = np.column_stack([np.ones(n), x, x**2, x**3,
                        h(x, 3.0), h(x, 6.0)])
fit = sm.OLS(y, X_sp).fit()
```

Kubisk spline med K interne knutepunkter har $K + 4$ basisfunksjoner. Natural spline er lineær i halene.

ISLP BSpline.

```
from ISLP.transforms import BSpline

bs = BSpline(internal_knots=[3.5, 5.5], degree=3, intercept=True)
Xbs = bs.fit_transform(x.reshape(-1, 1))
fit = sm.OLS(y, Xbs).fit()
```

11. GAM og backfitting

Generalized additive model. $y = b_0 + f_1(x_1) + f_2(x_2) + \dots + e$. Brukes når noen prediktorer har ikke-lineær sammenheng med y , men man fortsatt vil tolke effekter separat.

Finn kandidater til ikke-linearitet.

```
for i, col in enumerate(var_names):
    xi = X[:, i]
    r2_lin = sm.OLS(y, sm.add_constant(xi)).fit().rsquared
    Xq = sm.add_constant(np.column_stack([xi, xi**2]))
    r2_quad = sm.OLS(y, Xq).fit().rsquared
    print(col, round(r2_quad - r2_lin, 3))
```

B-spline brukt i GAM-lignende designmatrise.

```
bs = BSpline(internal_knots=[-1, 0, 1], degree=3, intercept=False)
X3_sp = bs.fit_transform(X[:, 2].reshape(-1, 1))
X_gam = np.column_stack([np.ones(len(y)), X[:, 0], X[:, 1],
                        X3_sp, X[:, 3:]])
gam = sm.OLS(y, X_gam).fit()
```

Backfitting, som i gam_example.py.

```
def knn(x0, x, y, K=20):
    d = np.abs(x - x0)
    return np.mean(y[np.argsort(d)[:K]])

b0 = y.mean()
yd = y - b0
K = 20

f1 = b0 + np.array([knn(x0, x1, yd, K) for x0 in x1])
res = y - f1
res = res - res.mean()
f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])

for i in range(10):
    res = y - f2
    res = res - res.mean()
    f1 = b0 + np.array([knn(x0, x1, res, K) for x0 in x1])

    res = y - f1
    res = res - res.mean()
    f2 = b0 + np.array([knn(x0, x2, res, K) for x0 in x2])
```

12. Trær, bagging, random forest og boosting

Decision tree. Rekursiv binær splitting. Regresjon: bladprediksjon er gjennomsnitt. Klassifikasjon: majoritetssklasse/sannsynlighet. Ubegrensede trær overfitter.

```
from sklearn.tree import DecisionTreeRegressor, DecisionTreeClassifier,
    plot_tree

tree = DecisionTreeRegressor(min_samples_leaf=30, random_state=123)
tree.fit(X_train, y_train)
plot_tree(tree, feature_names=X_train.columns, precision=2, fontsize=9)
pred = tree.predict(X_test)
MSE = np.mean((y_test - pred)**2)
```

Pruning med cost-complexity og CV.

```
big_tree = DecisionTreeRegressor(min_samples_leaf=2, random_state=0)
big_tree.fit(X_train, y_train)
path = big_tree.cost_complexity_pruning_path(X_train, y_train)
kfold = KFold(5, shuffle=True, random_state=10)
grid = GridSearchCV(big_tree, {"ccp_alpha": path.ccp_alphas},
```

```
    refit=True, cv=kfold,
    scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)
pruned_tree = grid.best_estimator_
```

Manuell første split.

```
from scipy.optimize import minimize_scalar

def RSS_split(s, x, y):
    R1 = y[x < s]
    R2 = y[x >= s]
    if len(R1) == 0 or len(R2) == 0:
        return np.inf
    return np.sum((R1 - R1.mean())**2) + np.sum((R2 - R2.mean())**2)
```

```
res = minimize_scalar(RSS_split, bounds=(x.min(), x.max()),
    args=(x, y), method="bounded")
```

Bagging og random forest.

```
from sklearn.ensemble import RandomForestRegressor,
    RandomForestClassifier
```

```
bag = RandomForestRegressor(n_estimators=500,
    max_features=X_train.shape[1],
    random_state=123)

bag.fit(X_train, y_train)

rf = RandomForestRegressor(n_estimators=500, max_features=3,
    random_state=123)

rf.fit(X_train, y_train)
```

```
pd.Series(rf.feature_importances_,
    index=X_train.columns).sort_values().plot.barh()
```

Bagging i sklearn er `RandomForestRegressor(max_features=p)`, der $p = X_{\text{train}}.shape[1]$. Random forest bruker færre prediktorer per split, f.eks. `max_features=3` i `Carseats`/`Ames`.

BART. Bayesian additive regression trees er Bayesiansk sum av trær og ligger i pensum/ISLP, men er mindre sentralt i forelesers eksamensnære Python-filer enn bagging, random forest og boosting.

Boosting.

```
from sklearn.ensemble import GradientBoostingRegressor,
    GradientBoostingClassifier
```

```
boost = GradientBoostingRegressor(n_estimators=1000,
    learning_rate=0.01,
    max_depth=2,
    random_state=123)
```

```
boost.fit(X_train, y_train)
```

```
gbc = GradientBoostingClassifier(n_estimators=1000,
    learning_rate=0.01,
    max_depth=2,
    random_state=123)
```

```
gbc.fit(X_train, y_train)
prob = gbc.predict_proba(X_test)[: , 1]
```

13. SVM og nevrle nettverk

SVM. I sklearn er C invers regulering: liten C gir sterkere regulering og bredere/mykere margin; stor C gir mer fleksibel grense. RBF-kernel bruker γ : stor γ gir mer lokal/kompleks grense.

```
from sklearn.svm import SVC
```

```
param_lin = {"C": [0.01, 0.1, 1, 10, 100, 1000]}
tune_lin = GridSearchCV(SVC(kernel="linear"), param_lin,
    cv=5, scoring="accuracy")

tune_lin.fit(X, y)
```

```
param_poly = {"C": [0.01, 1, 100, 1000], "degree": [1, 2, 3, 4]}
tune_poly = GridSearchCV(SVC(kernel="poly"), param_poly, cv=5)
tune_poly.fit(X, y)
```

```
param_rbf = {"C": [0.01, 1, 100, 1000], "gamma": [0.5, 1, 2, 4]}
tune_rbf = GridSearchCV(SVC(kernel="rbf"), param_rbf, cv=5)
tune_rbf.fit(X, y)
```

```
pred = tune_rbf.best_estimator_.predict(X_test)
```

Neural networks.

Foreleser bruker `MLPRegressor`/`MLPClassifier` fra sklearn, ikke `PyTorch`/`CNN`/`RNN`. Standardiser X , særlig for neural

```
networks.
from sklearn.neural_network import MLPRegressor, MLPClassifier

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

nn = MLPRegressor(hidden_layer_sizes=(15,), activation="relu",
                  max_iter=100000, random_state=0)
nn.fit(X_train_s, y_train)
pred = nn.predict(X_test_s)

dnn = MLPRegressor(hidden_layer_sizes=(15, 8), activation="relu",
                  max_iter=100000, random_state=0)
```

14. PCA og clustering

Unsupervised learning. Vi observerer bare X, ikke y. Brukes til eksplorativ analyse og hypoteser, ikke supervisert prediksjon.

PCA.

- PCA lager nye variabler Z1, Z2, ... som lineære kombinasjoner av X.
- Første principal component maksimerer varians.
- Andre component maksimerer gjenværende varians og er ukorrelert med første.
- Standardiser først når variabler har ulik skala.
- Loadings = vektorer for variablene. Scores = observasjonenes verdier på PC-ene.

```
from sklearn.decomposition import PCA

def summary_pca(fit):
    return pd.DataFrame({
        "Standard deviation": np.sqrt(fit.explained_variance_),
        "Proportion of Variance": fit.explained_variance_ratio_,
        "Cumulative Proportion":
            ↪ np.cumsum(fit.explained_variance_ratio_)
    })

scaler = StandardScaler(with_mean=True, with_std=True)
X_scaled = scaler.fit_transform(X)
pca = PCA()
scores = pca.fit_transform(X_scaled)
summary_pca(pca)

loadings = pd.DataFrame(
    pca.components_.T,
    index=X.columns,
    columns=[f"PC{i+1}" for i in range(pca.n_components_)])
)
```

K-means og hierarkisk clustering.

- K-means krever valgt K og minimerer within-cluster variation.
- Hierarkisk clustering gir dendrogram og trenger ikke forhåndsvalgt K.
- Linkage: `complete` = maks avstand, `single` = min, `average` = snitt, `ward` minimerer varians.
- Standardiser før avstandsbaserte metoder.

```
from sklearn.cluster import KMeans, AgglomerativeClustering
from scipy.cluster.hierarchy import dendrogram, cut_tree
from ISLP.cluster import compute_linkage

X_s = StandardScaler().fit_transform(X)

km = KMeans(n_clusters=3, random_state=1, n_init=10)
km.fit(X_s)
pd.crosstab(km.labels_, y_true)

hc = AgglomerativeClustering(n_clusters=3, linkage="complete")
labels = hc.fit_predict(X_s)
```

15. Typiske eksamenssetninger

Modellvalg. “Jeg velger parameteren med lavest kryssvalidert feil på treningsdata. Testdata brukes først etter at modellen er valgt.”

Regularisering. “Ridge og LASSO reduserer varians ved å straffe store koeffisienter. Ridge krymper koeffisienter, mens LASSO også kan sette dem til null.”

Klassifikasjon. “Accuracy alene kan være misvisende når klassene er ubalanserte, derfor ser jeg også på krysstabell og rad-proporsjoner.”

Prediksjon vs forklaring. “For prediksjon prioriteres testfeil. For forklaring ser jeg også på koeffisienter, p-verdier, usikkerhet og om modellen gir faglig mening.”

Lekkasje. “Variabler som inneholder informasjon som ikke er tilgjengelig ved prediksjonstidspunktet, eller som er direkte avledet fra responsen, må fjernes.”

Standardisering. “Scaler tilpasses på treningsdata og brukes deretter på testdata. Dette unngår informasjonslekkasje fra testsettet.”